```cpp
#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <set>
#include <algorithm>
#include <cctype>
#include <fstream>

#include "CetzModelo.hpp" // Cabecera actualizada

using namespace std;

struct Tokenizador {
    unordered_map<char, size_t> charToId;
    unordered_map<size_t, char> idToChar;
    size_t eosId;

    void construirVocabularioRobusto(const vector<string>& dataset) {
        set<char> caracteresUnicos;

        for (char c = 32; c <= 126; ++c) {
            caracteresUnicos.insert(c);
        }

        for (const string& texto : dataset) {
            for (char c : texto) caracteresUnicos.insert(c);
        }

        size_t id = 0;
        for (char c : caracteresUnicos) {
            charToId[c] = id;
            idToChar[id] = c;
            id++;
        }

        eosId = id;
        idToChar[eosId] = '$';
    }

    size_t vocabSize() const { return idToChar.size(); }

    vector<size_t> codificar(const string& texto) const {
        vector<size_t> ids;
        for (char c : texto) {
            if (charToId.count(c)) {
                ids.push_back(charToId.at(c));
            }
        }
        return ids;
    }

    string decodificar(size_t id) const {
        if (idToChar.count(id)) return string(1, idToChar.at(id));
        return "";
    }
};

vector<string> cargarCorpus(const string& ruta) {
    vector<string> dataset;
    ifstream in(ruta);
    if (!in.is_open()) return dataset;

    string linea;
    while (getline(in, linea)) {
        if (!linea.empty()) {
```

```
            for (char &c : linea) c = static_cast<char>(toupper(c));
            dataset.push_back(linea);
        }
    }
    in.close();
    return dataset;
}

int main() {
    string archivoCorpus = "corpus.txt";
    vector<string> dataset = cargarCorpus(archivoCorpus);

    if (dataset.empty()) {
        cout << "[AVISO]: No se encontró 'corpus.txt' o está vacío. Cargando dataset por
defecto..." << endl;
        dataset = {
            "HOLA COMO ESTAS",
            "ME LLAMO CETZ",
            "EL PERRO ES UN AMIGO",
            "CETZ ES UN MODELO DE LENGUAJE EN C++"
        };
    } else {
        cout << "¡Corpus cargado exitosamente! Total de frases: " << dataset.size() <<
endl;
    }

    Tokenizador tokenizador;
    tokenizador.construirVocabularioRobusto(dataset);

    size_t vocabSize = tokenizador.vocabSize();
    size_t dimModel = 16;
    size_t numHeads = 2;
    size_t dimFFN = 64;
    size_t numBloques = 2;
    size_t maxSeqLen = 60;

    // Instancia con la clase renombrada CetzModelo
    CetzModelo cetzModelo(vocabSize, dimModel, numHeads, dimFFN, numBloques, maxSeqLen);

    cout << "=================================================" << endl;
    cout << "   CETZ-MODELO DE LENGUAJE (Clase CetzModelo)" << endl;
    cout << "   - Vocabulario Robusto: " << vocabSize << " tokens (ASCII)" << endl;
    cout << "=================================================" << endl;
    cout << "1. Entrenar con " << archivoCorpus << " y guardar" << endl;
    cout << "2. Cargar 'cetz_corpus.bin' existente" << endl;
    cout << "Opción > ";

    int opcion = 1;
    cin >> opcion;
    cin.ignore();

    string archivoPesos = "cetz_corpus.bin";

    if (opcion == 1) {
        int epocas = 1000;
        double lr = 0.0025;
        int t = 1;

        cout << "\nEntrenando CetzModelo (" << epocas << " épocas)..." << endl;
        for (int e = 0; e < epocas; ++e) {
            for (const string& frase : dataset) {
                vector<size_t> seqEntrada = tokenizador.codificar(frase);
                if (seqEntrada.empty()) continue;

                vector<size_t> seqObjetivo = seqEntrada;
                seqObjetivo.erase(seqObjetivo.begin());
```

```
                seqObjetivo.push_back(tokenizador.eosId);

                cetzModelo.trainStep(seqEntrada, seqObjetivo, lr, t);
            }

            if ((e + 1) % 100 == 0 || e == 0) {
                cout << "  Progreso: Época " << (e + 1) << " / " << epocas
                     << " (" << ((e + 1) * 100 / epocas) << "%)" << endl;
            }
        }

        if (cetzModelo.guardar(archivoPesos)) {
            cout << "\n¡Modelo guardado exitosamente en '" << archivoPesos << "'!" <<
endl;
        }
    } else {
        if (cetzModelo.cargar(archivoPesos)) {
            cout << "\n¡Modelo cargado exitosamente desde '" << archivoPesos << "'!" <<
endl;
        } else {
            cout << "[ERROR]: No se pudo abrir '" << archivoPesos << "'." << endl;
            return 1;
        }
    }

    cout << "\n================================================" << endl;
    cout << "   INTERFAZ CETZ-LM (Escribe cualquier texto)" << endl;
    cout << "================================================\n" << endl;

    double temperatura = 0.6;
    int topK = 4;
    double topP = 0.9;
    double penalizacionRepeticion = 1.4;

    string entrada;
    while (true) {
        cout << "Prompt para Cetz > ";
        if (!getline(cin, entrada) || entrada == "salir") break;
        if (entrada.empty()) continue;

        for (char &c : entrada) c = static_cast<char>(toupper(c));

        vector<size_t> contexto = tokenizador.codificar(entrada);
        cout << "Cetz-LM -> ";

        for (int step = 0; step < 30; ++step) {
            size_t idSiguiente = cetzModelo.predecirSiguiente(
                contexto, temperatura, topK, topP, penalizacionRepeticion
            );

            if (idSiguiente == tokenizador.eosId) break;

            contexto.push_back(idSiguiente);
        }

        for (size_t id : contexto) {
            cout << tokenizador.decodificar(id);
        }
        cout << "\n------------------------------------------------" << endl;
    }

    return 0;
}
```